트램 전방 충돌 위험 알림 시스템

라즈베리파이 5 온디바이스 판단 + 노트북 검출 분담 · 최종 계획

안전 주의 경고 위험

동아리 트램 캡스톤 · 2026.10

피할 수 없는 트램을 위해 , 몇 초 먼저 알려 준다

문제

- 레일 위를 달려 피해 갈 수 없음
- 제동거리가 김 (50 km/h → 약 94 m)
- 선로 옆 보행자가 많아 ' 있다 ' 만으론 경보가 계속 울림

목표

- 카메라 하나로 선로 위 사람 · 차량 검출
- 거리와 충돌까지 남은 시간 (TTC) 계산
- 안전 · 주의 · 경고 · 위험 4 단계로 경보

최소 성능 목표 (1:10 모형)

10 FPS
초당 판단 횟수

0.3 초
촬영 → 경보 지연

5 m
감지 거리 (실제 50 m)

0 건
위험 상황 놓침

보드 제외 약 22 만원 , 주행체 포함해도 30 만원 안

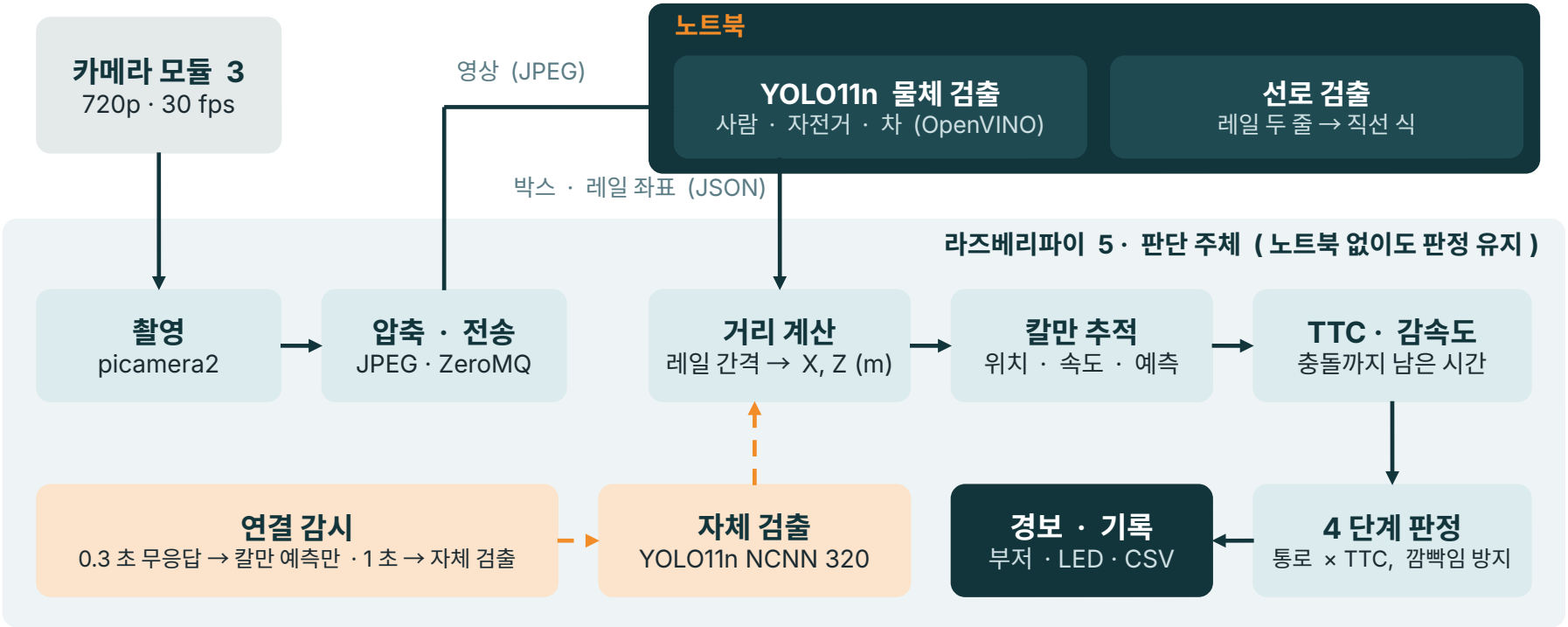
품목	쓰는 곳	예산 (원)
라즈베리파이 카메라 모듈 3 + 파이 5 용 케이블	전방 촬영 (화각 약 66°, HDR)	50,000
공식 27W 전원 · 액티브 쿨러 · 케이스	파이 안정 동작 , 발열 대비	40,000
microSD 64 GB (A2)	OS · 로그 저장	15,000
5 GHz 공유기	파이 ↔ 노트북 전용 무선망	30,000
보조배터리 (USB-C PD, 5V 3A 이상)	주행체 위 파이 전원	40,000
부저 · LED · 점퍼선	소리 · 빛 경보	10,000
하드보드지 · 테이프 · 피규어 · 눈금	카메라 거치대 , 1:10 선로판	30,000
필수 합계		**215,000**
선택 : 주행체 (RC 카 + 엔코더)	일정 속도 주행 , 속도 정답	80,000

가격은 추정치 · 발주 전 국내 판매가 확인

이미 있는 것

- 라즈베리파이 5 (8GB) · 보드 예산 별도
- 노트북 Core Ultra 7, Arc GPU
- 팀원 휴대폰 (초기 영상)

판단은 파이가 , 노트북은 무거운 검출만 돕는다



7 단계 처리 흐름 : 무엇을 넣어 무엇을 얻나

단계	입력 → 출력	방법	장치	예제
① 촬영 · 전송	카메라 → JPEG 사진	picamera2, ZeroMQ	파이	10
② 선로 검출	사진 → 레일 두 줄의 직선 식	테이프 색 분리 + 직선 맞춤	노트북 *	03
③ 물체 검출	사진 → 사람 · 차 박스	YOLO11n (사전학습 → 미세조정)	노트북 *	02 · 09
④ 거리	박스 발끝 + 레일 간격 → 옆 X, 앞 Z (m)	궤간 1.435 m 비례 + 사람 키 보조	파이	04
⑤ 추적 · 예측	X, Z → 위치 · 속도 (+ 불확실성)	칼만 필터 (등속 모델) + 게이트 매칭	파이	07
⑥ 판단	위치 · 속도 → TTC, 필요 감속도 , 등급	규칙표 + 깜빡임 방지	파이	05
⑦ 경보 · 기록	등급 → 부저 · LED · 로그	gpiozero, CSV	파이	10

* 연결이 끊기면 파이가 직접 (선로는 원래 가벼움 , 검출은 NCNN 320 경량 모델)

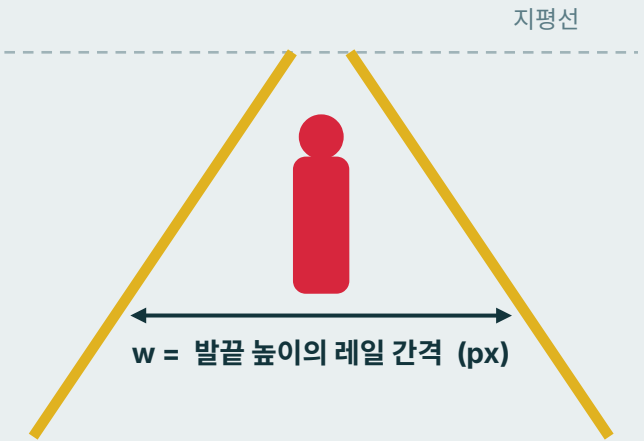
안전
통로 밖 , 멀어지거나 그대로

주의
통로 안이지만 멀다 / 통로 1 m 안쪽

경고
TTC 5 초 이하 / 통로로 들어오는 중

위험
TTC 3 초 이하 / 필요 감속도 > 1.2 m/s²

레일 간격이 ' 자 ' 가 되어 절대 거리 (m) 를 잰다



Z = fx × 궤간 ÷ w

fx: 카메라 보정으로 구한 초점거리 (px)
궤간 : 1.435 m (모형 14.35 cm, 이미 아는 길이)

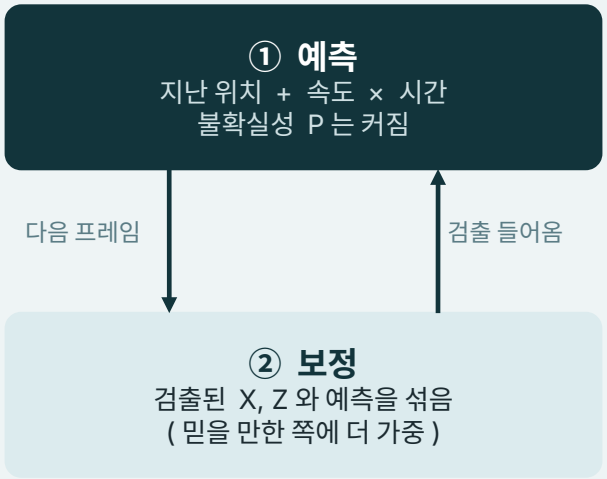
왜 상대 거리가 아닌가

- 카메라 하나는 크기만 보면 ' 상대적 ' 이지만 , 길이를 아는 물체 (레일 간격) 가 같은 높이에 있으면 실제 m 로 환산됨
- fx 는 체커보드 20 장 보정 (OpenCV) 으로 한 번 구함
- 백업 : 사람 키 (1.7 m) · 카메라 높이로 바닥 평면 계산

모형 거리	레일 간격	1 px 틀리면	실제 환산
1 m	141 px	±0.7 cm (0.7%)	10 m
2 m	71 px	±2.8 cm (1.4%)	20 m
3 m	47 px	±6.4 cm (2.1%)	30 m
5 m	28 px	±18 cm (3.5%)	50 m

카메라 모듈 3, 1280 px, 화각 66° 가정 (fx ≈ 985 px) · 검증 : 선로판 50 cm 마다 눈금과 비교

미터 좌표로 바꾼 뒤라 ' 선형 칼만 + 등속 모델 ' 이면 충분



연결이 끊기면 ②를 건너뛰고 ①만 반복 → 끊김 대응에 그대로 쓰임

종류	언제 쓰나	우리 판단
선형 KF	측정과 상태가 선형 관계	**선택 (측정을 m 로 바꿈)**
EKF	픽셀을 바로 측정으로 쓸 때	불필요
UKF	비선형이 강할 때 · 계산 큼	불필요
IMM	걷기 · 멈춤 모델 섞기	확장 목표 (급정지)
파티클	분포가 복잡할 때 · 무거움	불필요

상태 [X, Z, Vx, Vz] **측정 잡음 R** 멀수록 크게 ($\sigma Z \propto Z^2$)
예제 07 결과 거리 오차 0.99 → 0.51 m, 예측 1 회 약 11 µs (PC)

참고 : SORT 식 화면 박스 칼만 ([u, v, s, r]) 은 노트북 쪽 ByteTrack 안에 이미 있음

검증된 방법을 조합하고 , 판단 규칙을 우리가 설계

기술	참고 문헌	우리가 쓰는 곳
YOLO	Redmon et al., "You Only Look Once", CVPR 2016 · Ultralytics YOLO11	사람 · 차 검출
칼만 필터	Kalman, "A New Approach to Linear Filtering and Prediction Problems", 1960	추적 · 끊김 시 예측
SORT	Bewley et al., "Simple Online and Realtime Tracking", ICIP 2016	추적 구조
ByteTrack	Zhang et al., ECCV 2022	노트북 쪽 화면 추적
TTC	Lee, "A theory of visual control of braking...", Perception 1976	충돌까지 남은 시간
RailSem19	Zendel et al., CVPR Workshops 2019	실제 레일 분할 학습 (향후)
RailSafeNet	오픈소스 (MIT), 트램 레일 + 궤간 기반 거리 구역	옆 거리 구역 아이디어

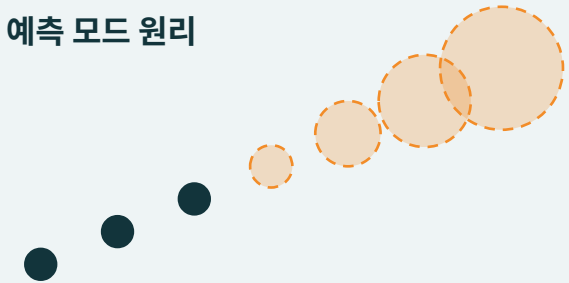
우리가 새로 만드는 것 : 통로 × TTC × 필요 감속도 등급 규칙 , 연결 끊김 대응과 그 검증

노트북과 끊기면 : 바로 예측 , 1 초 뒤엔 파이가 직접 본다

연결이 끊긴 시각부터 (발표 2 의 0.3 초 ' 유지 ' 단계는 시뮬레이션 결과로 삭제)

정상	0 ~ 1 초	1 초 ~
노트북 검출 + 파이 판단	**예측 모드**	**자체 검출 모드**
평소에도 파이 자체 모델은 메모리에 올려 둠	칼만 예측으로 판정 , 불확실성 2σ 만큼 통로를 넓힘 등급은 올리기만 , 내리지 않음	파이가 YOLO11n(NCNN, 320) 직접 실행 ' 감시 저하 ' 표시 + 재연결 시도

예측 모드 원리

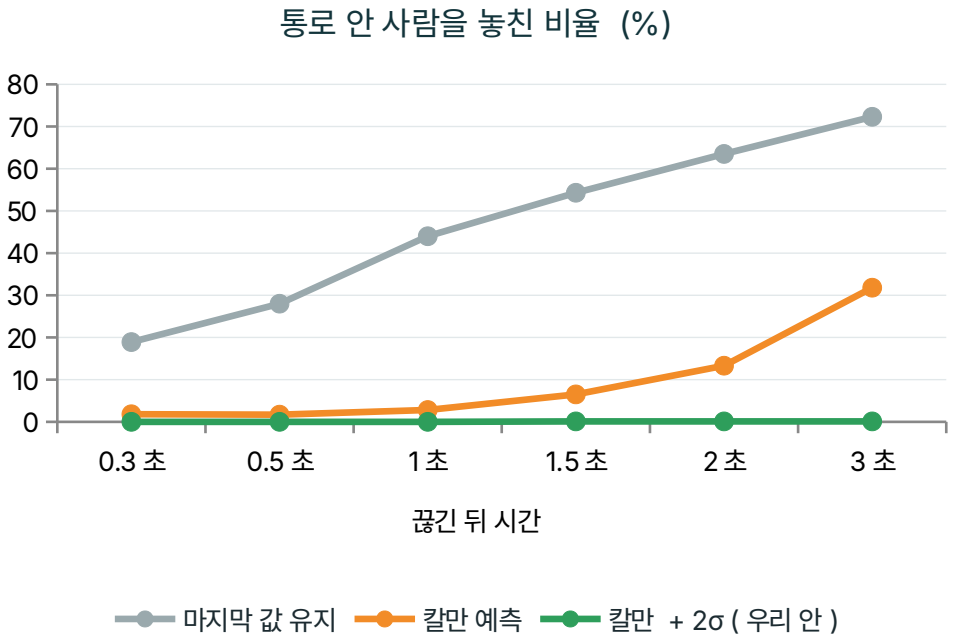


● 끊기기 전 검출 ○ 예측 (갈수록 불확실 = 원이 커짐)

그 밖의 문제

- 무선 지연 → 전용 5 GHz 공유기 , 0.3 초 넘은 결과 버림
- 먼 거리 오차 → 칼만 평활 + 거리별 잡음 반영
- 경보 깜빡임 → 올릴 땐 2 프레임 , 내릴 땐 1 초 유지
- 파이 발열 → 액티브 쿨러 , 온도 로그

시뮬레이션 : 예측은 1 초까지 유효 , 그 뒤는 직접 검출



끊긴 뒤	앞 거리 오차 (칼만)	헛경보 (칼만 +2σ)
0.3 초	0.30 m	14%
0.5 초	0.36 m	23%
1 초	**0.50 m**	**58%**
1.5 초	0.63 m	91%
2 초	0.78 m	99%

오차는 중앙값 , 마지막 값 유지는 0.3 초에 2.6 m

- 3,000 회 중 유지는 0.3 초에 19% 놓침 → 바로 예측
- 칼만 +2σ 는 놓침 0% 지만 1 초 넘으면 헛경보 급증
- → 1 초부터 파이 자체 검출이 꼭 필요 (새로 나타난 사람도 못 봄)

예제 08 · 실제 크기 , 트램 20~40 km/h, 보행자 걷기 50% / 급정지 25% / 되돌아감 25% 가정 · 모형 시험 (S9) 으로 재확인

사전학습 YOLO 로 먼저 재 보고 , 부족하면 미세조정

① 수집

시험장 영상에서
1 초에 1~2 장 추출
300~500 장

② 라벨링

Roboflow 또는 CVAT
YOLO 형식
사람 · 차 · 자전거

③ 학습

COCO 사전학습
YOLO11n 전이학습
Colab GPU

④ 검증

mAP50, 거리별
재현율 (놓침)
학습에 안 쓴 사진

⑤ 변환

노트북 : OpenVINO
파이 : NCNN 320
(INT8 선택)

실제 명령 (Ultralytics)

```
yolo train model=yolo11n.pt data=tram.yaml epochs=100
imgsz=640
yolo val   model=best.pt data=tram.yaml
yolo export model=best.pt format=openvino
yolo export model=best.pt format=ncnn imgsz=320
```

왜 이렇게

- 피규어는 진짜 사람과 달라 COCO 모델이 놓칠 수 있음
- 3 주차 재현율 90% 미만이면 미세조정
- 선로는 모형에선 색 분리 (학습 불필요)

이번 주 지연 측정부터 , 7 주차 모형 실시간 경보

2 주 · 이번 주 | 3~4 주 | 5~7 주 | 8~10 주

준비 · 관문

부품 발주
파이↔노트북 지연 측정
예제 07~10 시험

기초 구현

선로 · 거리 · 등급
카메라 보정 · 시험장
사전학습 성능 측정

온디바이스 통합

파이 추적 · 판단 · 경보
끊김 대응 시험
필요 시 미세조정

시험 · 발표

시나리오 S1~S9 시험
분담 vs 파이 단독 비교
최종 발표

2 주 왕복 지연 0.3 초 이내 확인

4 주 PC 데모

7 주 모형 실시간 경보 (MVP)

10 주 최종 발표

Plan B: 무선 지연이 크면 주행 시험만 유선 연결 , 그래도 안 되면 파이 단독 검출로 전환

부록 · 예상 질문

거리 측정 · 칼만 필터 · 모델 학습 · 예측 모드와 자체 검출 · 활용 코드

Q1 거리는 상대적인데 어떻게 재나 ?

세 가지 방법을 섞고 , 눈금으로 실제 오차를 잰다

방법	식	필요한 것	약점
A. 레일 간격 (주)	Z = fx · 궤간 ÷ w	레일 검출 , fx	레일이 가려지면 못 씀
B. 바닥 평면	Z = h · fy ÷ (y – y 지평선)	카메라 높이 h, 기울기	경사 · 흔들림에 민감
C. 물체 크기	Z = fy · 키 ÷ 박스 높이	물체 실제 키	키가 사람마다 다름

카메라 보정 절차

- A4 체커보드를 여러 각도로 20 장 촬영
- cv2.calibrateCamera → fx, fy, 렌즈 왜곡
- 왜곡 보정 후 거리 계산 (가장자리 오차 감소)

검증 방법

- 선로판에 50 cm 마다 눈금 , 피규어를 놓고 측정값 vs 정답
- 거리 구간별 평균 오차 · 최대 오차를 표로 보고
- 목표 : 모형 3 m 안 오차 5% 이하 (제안)

세 값은 신뢰도 순 (A → B → C) 으로 고르고 , 칼만 필터가 시간에 따라 다시 평활 · 확장 : 파이 5 의 두 번째 카메라 단자로 스테레오

상태 4 개 , 행렬 4 개면 끝나는 작은 계산

항목	값	뜻
상태 x	[X, Z, Vx, Vz]	옆 · 앞 위치 (m) 와 상대 속도 (m/s)
전이 F	X ← X + Vx·Δt, Z ← Z + Vz·Δt	등속도로 움직인다고 가정
측정 H	[I 0]	위치 X, Z 만 측정
과정 잡음 Q	가속도 σa = 1 m/s²	급정지 · 방향 전환 여유
측정 잡음 R	σZ = Z² ÷ (fx · 궤간) × 1.5 px	멀수록 거리 측정을 덜 믿음
짝 맞추기	마할라노비스 거리² < 9.21	새 검출이 어느 물체 것인지 (99%)
삭제	1.5 초 동안 측정 없으면	사라진 물체 정리

왜 화면 좌표가 아니라 미터 좌표에서 ? 예측 결과가 곧바로 'TTC' 와 ' 통로 안 / 밖 ' 판단에 쓰이기 때문 · 라이브러리 : numpy 만 (filterpy, cv2.KalmanFilter 로도 가능)

데이터 · 평가 기준을 미리 정해 두고 학습한다

항목	계획
데이터	시험장 영상 프레임 300~500 장 , 조명 3 종 · 거리 1~5 m · 가림 포함 , 물체 없는 사진 10%
나누기	학습 70% · 검증 20% · 시험 10% (같은 영상의 앞뒤 프레임이 섞이지 않게 영상 단위로)
라벨	person, car, bicycle (COCO 번호 유지) · Roboflow/CVAT · 2 명이 교차 확인
학습	YOLO11n 사전학습 가중치에서 시작 , epochs 100, imgsz 640, 기본 증강 · Colab 무료 GPU
평가	mAP50 · 거리 구간별 재현율 (놓침) · 노트북 / 파이 FPS
합격선 (제안)	모형 3 m 안 사람 재현율 95% 이상 , 파이 NCNN 320 에서 5 FPS 이상
실제 선로 (향후)	RailSem19 로 YOLO11n-seg 레일 분할 학습

라이선스 : Ultralytics AGPL-3.0 (공개 저장소로 쓰면 문제없음), RailSem19 는 연구용 이용 조건 확인

Q4 예측 모드와 파이 자체 검출 , 꼭 필요한가 ?

예측은 ' 이미 본 물체 ' 만 , 자체 검출은 ' 새 물체 ' 까지 본다

	칼만 예측 모드	파이 자체 검출
볼 수 있는 것	끊기기 전에 추적하던 물체	새로 나타난 물체 포함 전부
계산량	물체당 수십 µs	프레임당 수십 ~100 ms (추정)
유효 시간	약 1 초 (시뮬레이션)	제한 없음 (정확도는 낮음)
약점	급정지 · 방향 전환 , 새 물체	작은 입력이라 먼 물체 놓침
역할	끊긴 직후 공백 메우기	1 초 이후 감시 유지 + ' 감시 저하 ' 경보

둘 다 필요한 이유

- 자체 검출을 켜는 데 걸리는 시간 (모델 로딩 수 초) 을 예측 모드가 메움 → 모델은 평소에도 올려 둠
- 선택안 : 평소에도 파이가 초당 2~3 장 자체 검출 → 노트북 결과와 교차 확인

우리 예제 11 개 + 검증된 오픈소스

우리 예제 (code_examples)	내용
00 · 01	정답 있는 합성 영상 , OpenCV 기초
02 · 03	YOLO 검출 · 추적 , 테이프 선로 검출
04 · 05	거리 계산 , TTC · 4 단계 판정
06	PC 전체 파이프라인 데모
07 · 08	칼만 추적기 , 연결 끊김 시뮬레이션
09 · 10	노트북 서버 ↔ 파이 클라이언트 (끊김 대응 포함)

오픈소스	쓰는 곳	라이선스
Ultralytics YOLO	학습 · 추적 · 변환 , 파이 가이드	AGPL-3.0
picamera2	파이 카메라 촬영	BSD-2
pyzmq (ZeroMQ)	영상 · 결과 전송	BSD
OpenCV	보정 · 선로 · 그리기	Apache-2.0
filterpy	칼만 필터 참고 구현	MIT
SORT (abewley)	추적 구조 원본	GPL-3.0
gpiozero	부저 · LED	BSD
RailSafeNet	레일 · 거리 구역 아이디어	MIT

예제 07~10 은 PC 에서 실행 확인 완료 (파이 실기 시험은 보드 도착 후) · 라이선스는 사용 전 다시 확인